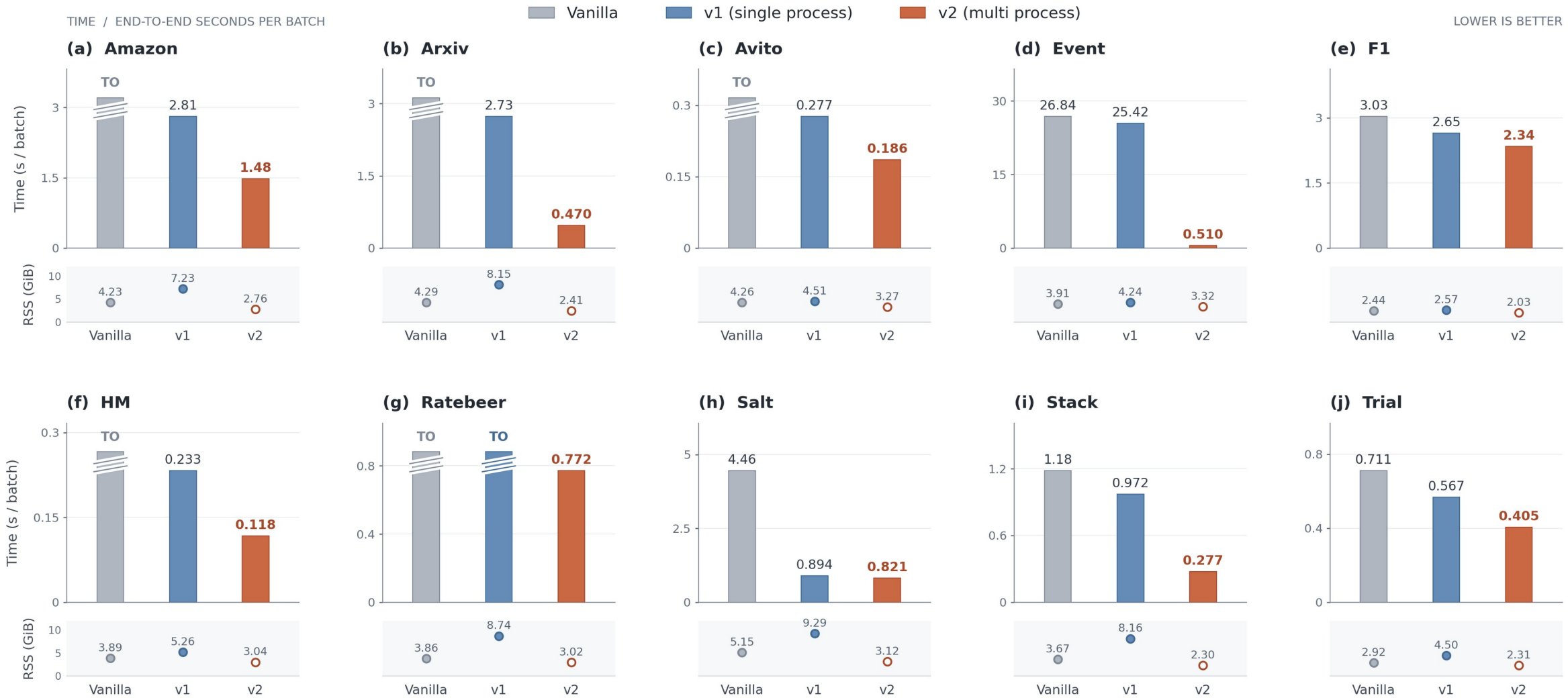
十数据集性能对比 | Vanilla · v1 · v2



逐数据集解释（1/2）｜瓶颈决定优化方式

左侧倍率： v1 / v2 的单位 batch 耗时；右侧列出证据与解释边界。

Amazon **1.89×**	**大量文本与特征供给限制训练吞吐** 并行组装＋持久化 embedding，减少重复文本模型调用。	部分热缓存； worker 诊断缺失， 不能量化各优化的独立贡献。
Arxiv **5.82×**	**长文本、低复用，组装速度拖慢 GPU** direct GloVe ＋并行组装，磁盘向量跨轮复用。	模型输入： 184,724 → 1,316 ； 运行前已有部分磁盘缓存。
Avito **1.49×**	**SQL 与编码缓存访问是主要供给成本** 多进程重叠采样 / 组装 / 训练，复用文本与编码特征。	第二轮文本模型输入为 0 ； 仍有 544.8 秒 ready 等待。
Event **49.83×***	**宽表初始化很重，训练批数却很少** 预算分组扫描＋精确分位数；缓存避免重复初始化。	* 图中是热初始化结果；冷测 2,164 → 606 秒才是独立证据。
F1 **1.13×**	**数据小，启动与进程协调占比高** 通用缓存复用；门槛避免不必要的磁盘持久化。	改善有限； 28.11 秒为最新验证， 没有证据表明新增初始化优化提速。

RSS 图显示的是记录值； v2 仅主进程，不能据此断言整体内存接近或降低。

逐数据集解释（2/2）｜减少重复工作与命中开销

左侧倍率： v1 / v2 的单位 batch 耗时；右侧列出证据与解释边界。

HM **1.98×**	**共享编码缓存命中后仍传输整块数据** 改为返回请求行；本次使用 worker 本地编码缓存。	4 worker 总预算 4 GiB； ready 等待 3,348.9 → 481.9 秒。
Ratebeer **≈1.19×**	**工作量大，仍持续消耗 SQL 与文本编码** 多进程重叠供给；共享缓存发布修复保障长跑稳定。	不是 4 小时内完成完整两轮； 详见下一页的规模与预算对照。
Salt **1.09×**	**高文本复用，瓶颈转为共享查询与锁** worker 本地文本→槽位索引，批量提取共享向量。	相对早期 v2： 18,260 → 5,208 秒； 相对已经优化的 v1，收益较小。
Stack **3.51×**	**共享查询竞争，以及大量磁盘分片映射** 批量查询 / 发布、锁外处理， mmap LRU 控制映射数。	复用既有磁盘 embedding； 不能把全部差异归因于并行。
Trial **1.40×**	**命中后仍有共享查询与磁盘访问成本** adaptive 本地索引＋已存在的磁盘 embedding 复用。	两轮文本模型输入均为 0； 此次改善包含热缓存收益。

RSS 图显示的是记录值； v2 仅主进程，不能据此断言整体内存接近或降低。

Ratebeer ｜为何两个 TO ，只有 v2 有完成时间?

图中“完成”同时受吞吐、运行规模和预算影响，需要拆开解释。

Vanilla

4 小时超时

目标： 1 × 20,743 批
缺少可靠的完成批数记录

v1 (single process)

4 小时超时

完成： 15,686 / 20,743 批
观测吞吐约 0.918 秒 / batch

v2 (multi process)

4 小时 17 分完成

完成： 2 × 10,000 批（截断）
端到端 0.772 秒 / batch

性能：观测吞吐约快 1.19× ，单位 batch 耗时约少 15.9% 。

机制：并行采样、 SQL / 特征组装与 GPU 训练，改善供给；但 ready 等待仍达 11,389 秒，
两轮仍各有约 2,000 万条文本模型输入，说明 SQL 与编码瓶颈尚未消除。

稳定性：先将 TensorFrame 存储转为共享内存，再发布缓存条目，修复并发读取损坏；
修复后通过本次 20,000 批长跑。这是正确性证据，不能独立解释 1.19× 的性能收益。

结论： v2 在本次配置下更快且成功跑完；尚未证明在同一 4 小时预算完成完整两轮。